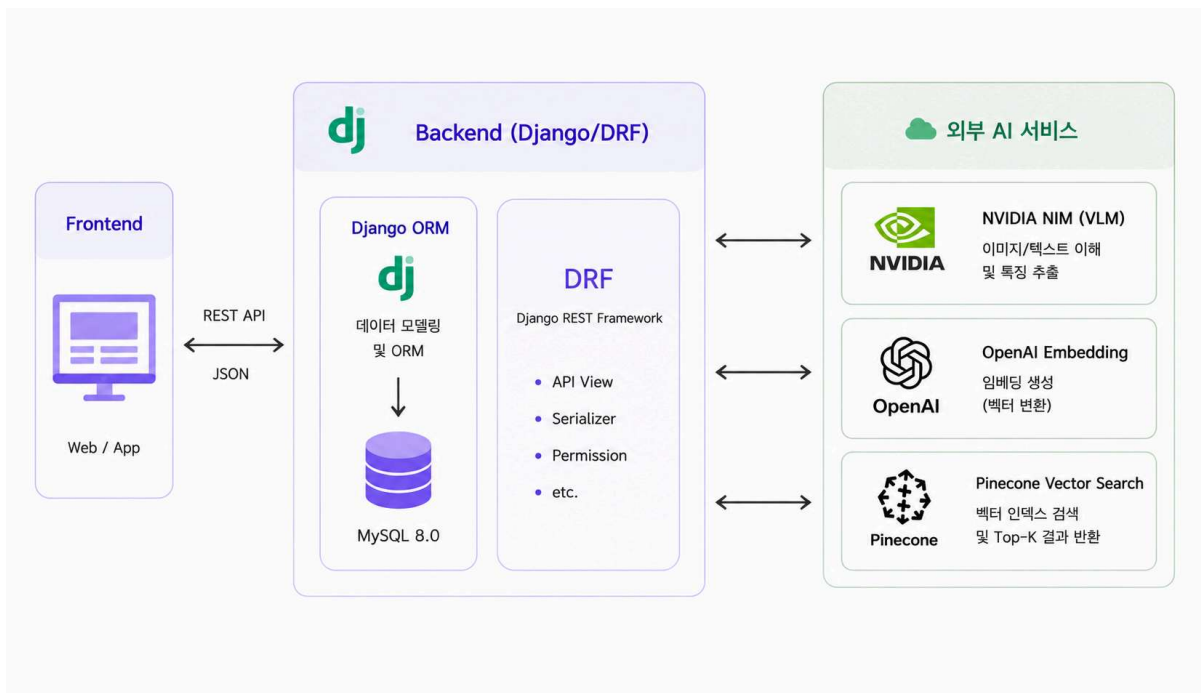


시스템 구성도

본 문서는 Olfit 프로젝트의 기술 아키텍처, 실행 흐름, 파이프라인 및 API 명세를 통합한 기술 명세서입니다.

1. 시스템 아키텍처 (System Architecture)

Olfit은 컨테이너 기반 아키텍처를 채택하고 있으며, 다수의 외부 서비스를 호출하여 사용하고 있습니다.



1.1 서비스 토폴로지 (Service Topology)

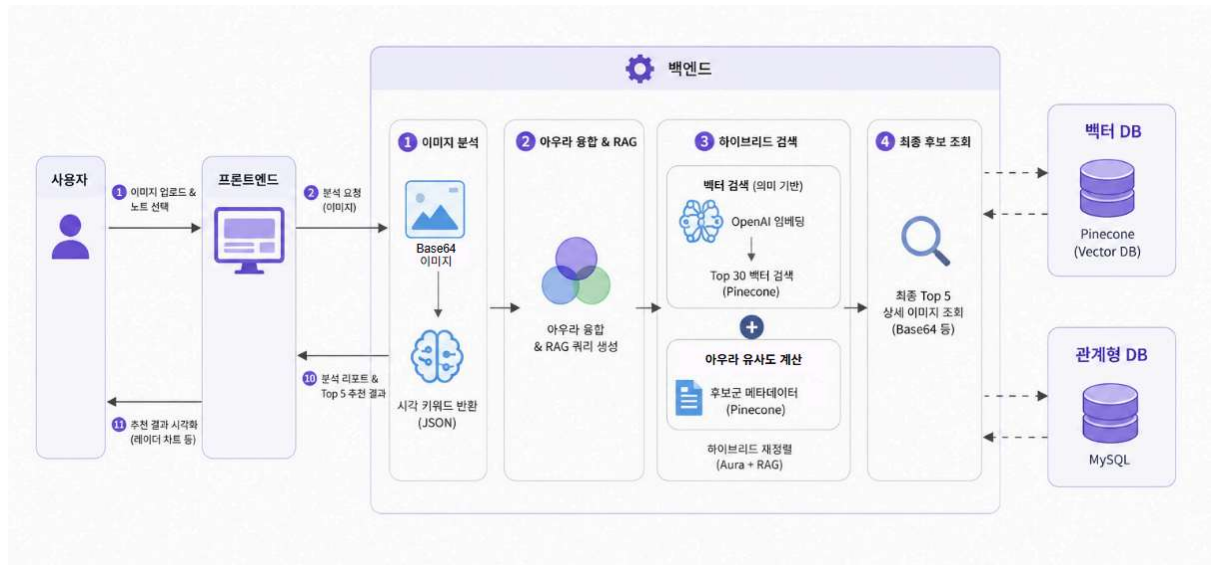
- **Frontend:** Vite + React 기반의 사용자 인터페이스.
- **Backend (Django):** API 서버 및 비즈니스 로직, AI 파이프라인 제어.
- **Database (MySQL):** 함수 메타데이터, 아우라 프로필, 이미지 에셋 영속화.
- **Vector Store (Pinecone):** 1536차원 벡터 기반의 실시간 시맨틱 검색.
- **AI Endpoints:**
 - **NVIDIA NIM:** VLM (google/gemma-3n-e4b-it)을 통한 이미지 분석.
 - **OpenAI API:** 텍스트 쿼리의 임베딩(text-embedding-3-small) 생성.

1.2 핵심 인프라 구성

- **Container:** Docker Compose를 통한 백엔드 및 데이터베이스 격리 운영.
- **Persistence:** 함수 메타데이터(MySQL)와 고차원 벡터 데이터(Pinecone)의 하이브리드 영속성 전략.
- **Security:** `.env` 기반의 환경 변수 관리 및 익명 세션 기반 프라이버시 보호.

2. 시퀀스 다이어그램 (Sequence Diagram)

Ofit 서비스의 전체 실행 순서는 다음과 같습니다.



분석 요청: 사용자가 업로드한 이미지(Base64)와 선택된 노트 리스트를 담아 `/api/analyze/` 호출.

이미지 분석: NVIDIA NIM API를 통해 업로드된 이미지를 분석하고, 스타일, 색상, 사물 등 시각 키워드가 담긴 JSON 응답 반환.

아우라 융합 & RAG: 추출된 시각 데이터와 사용자가 선택한 노트 정보를 결합해 5축 아우라 벡터를 산출하고, 벡터 검색을 위한 RAG 쿼리 생성

하이브리드 검색(벡터 + 키워드)

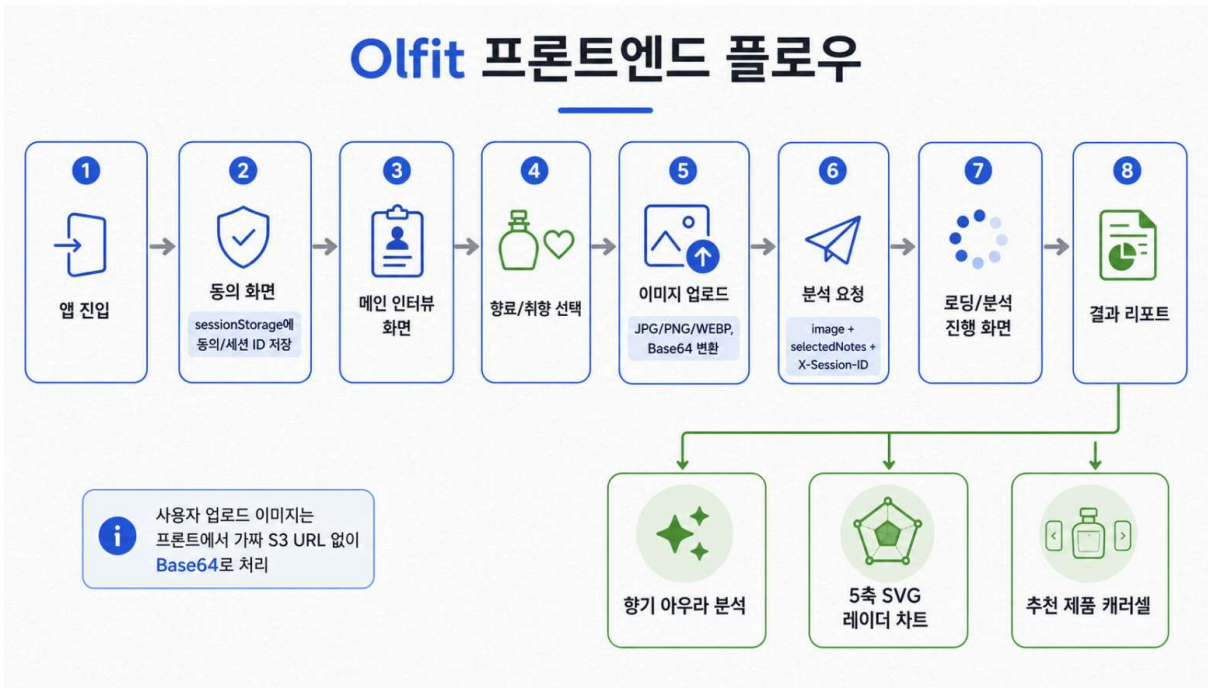
- **벡터 검색:** 검색 쿼리를 OpenAI 임베딩을 통해 dense 벡터로 변환한 뒤, Pinecone DB에서 벡터 검색을 수행해 유사도순으로 Top 30을 **1차 후보군**으로 선택
- **아우라 유사도 계산:** **1차 후보군** 모두에 대해 아우라 벡터의 **코사인 유사도**를 계산
- **하이브리드 재정렬:** 아우라 유사도와 벡터 유사도, 사용자가 선택한 노트의 포함 여부를 모두 고려하여 최종 하이브리드 재정렬을 진행합니다.

최종 후보 조회: 선정된 Top 5 항목의 ****상세 이미지(Base64 등)****를 DB에서 일괄 조회합니다. (텍스트 정보는 Pinecone 메타데이터를 활용하여 DB 부하를 최소화합니다.)

분석 결과 반환: 최종 확정된 Top 5 추천 항목 데이터와 아우라 분석 리포트 결과를 프론트엔드로 반환하고, 사용자 화면에 레이더 차트 등 시각화 요소와 함께 최종 분석 결과를 렌더링

3. 프론트엔드 실행 플로우 (Frontend Flow)

사용자와의 접점에서 상태 관리와 컴포넌트 생명주기를 통해 프로세스를 제어합니다.

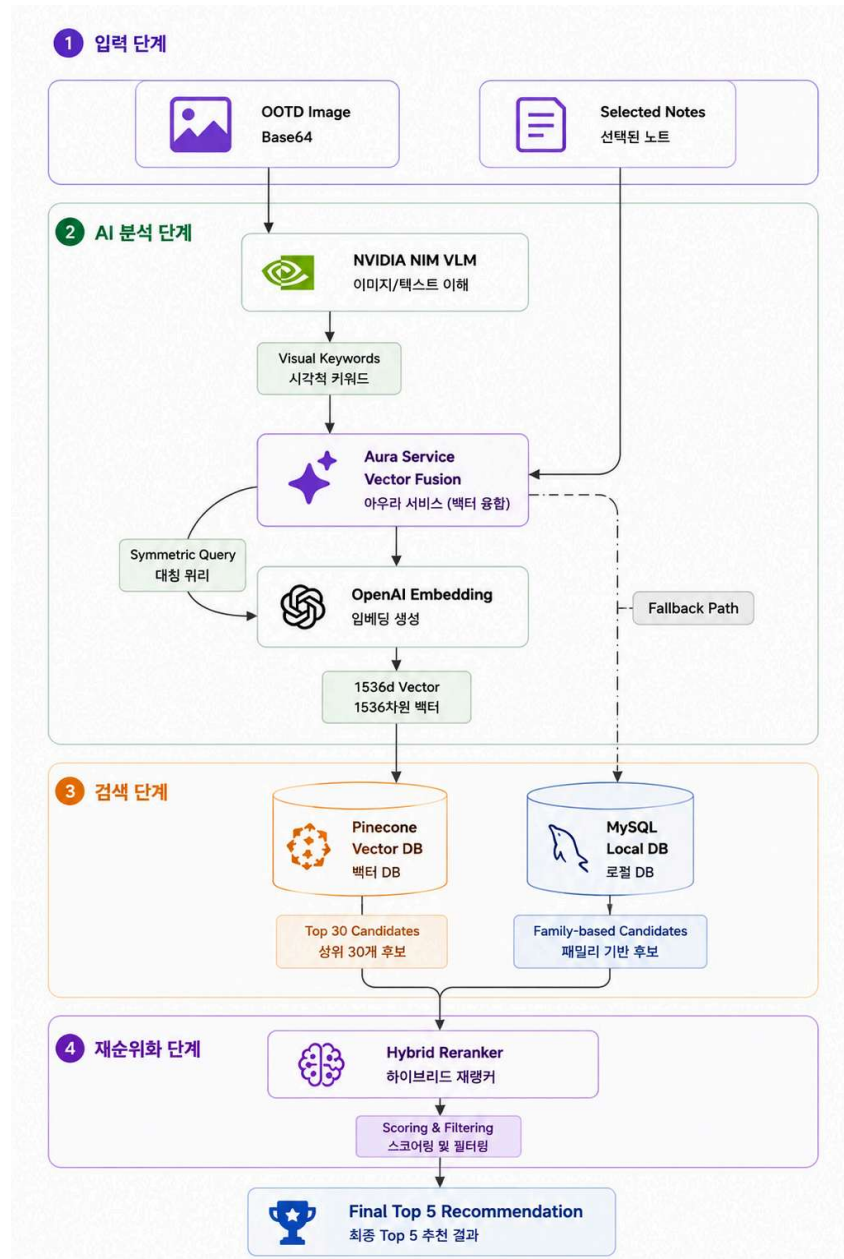


3.1 화면 및 데이터 흐름

1. 초기화 (Initialization): 개인정보 동의 후 익명 세션 ID 발급 및 Zustand 상태 저장.
2. 입력 (Collection): 사용자의 선호 성분 선택 및 이미지 업로드(Base64 변환).
3. 요청 (Dispatch): 분석 요청 API 호출 및 VLM 분석 애니메이션>Loading Screen) 활성화.
4. 결과 (Rendering): 백엔드 응답 매핑, 레이더 차트 시각화 및 리포트 캡처 기능 제공.

4. 백엔드 플로우 및 AI 파이프라인 (Backend Flow & AI Pipeline)

데이터의 변환과 AI 로직이 결합된 핵심 처리 공정인 **Symmetric Aura-RAG** 파이프라인입니다.



주요 단계별 흐름

- Request:** 사용자가 OOTD 이미지와 선호 향료를 전송.
- Vision Stage:** **VLEngine**이 이미지를 NVIDIA NIM으로 전달하여 시각적 특징(JSON) 추출.
- Aura Stage:**
 - 시각 키워드를 5차원 수치 데이터로 변환.
 - 사용자 취향(선호 노트)과 6:4 비율로 융합하여 **5축 아우라 벡터** 생성.
 - 시맨틱 검색을 위한 **대칭형 RAG 쿼리(Natural Language)** 생성.
- Retrieval Stage:**
 - 생성된 쿼리를 OpenAI로 임베딩 후 Pinecone에서 상위 30개 후보군 검색.
 - (Fallback) 외부 API 장애 시 MySQL에서 메인 계열 기반으로 후보군 추출.
- Reranking Stage:** 아우라 유사도, RAG 점수, 노트 가산점을 합산하여 최종 Top 5 재정렬 및 결과 반환.

4-1. AI 파이프라인 설계 (AI Pipeline)

VLM 프롬프트 전략

- **역할:** 전문 이미지 분석가 페르소나 부여.
- **제약:** 할루시네이션 방지를 위해 실제 보이는 요소에만 집중하며, 반드시 JSON 형식으로만 응답하도록 제어.
- **추출 필드:** 스타일 요약(visual_summary), 색상/사물(colors/objects), 분위기(mood/scene) 등.

대칭형 RAG (Symmetric RAG)

임베딩 모델의 성능 극대화를 위해, 검색 쿼리를 Pinecone에 저장된 향수 문서 구조와 동일한 문장 성분(시각 요약 + 무드 + 성분)으로 구성하여 검색 정확도를 높였습니다.

5. API 명세 (API Specification)

5.1 핵심 엔드포인트: 통합 분석 및 추천

- **Endpoint:** `POST /api/analyze/`
- **인증:** Header `X-Session-ID`

요청 (Request)

```
{
  "image": "data:image/jpeg;base64,...",
  "selectedNotes": ["베르가못", "머스크"]
}
```

응답 (Response)

```
{
  "personalMood": "#포근한 #우아한 #앰버",
  "fashionStyle": "포근한 베이지색 니트를 입은 모습",
  "radarScores": { "플로랄": 0.2, "우디": 0.5, ... },
  "recommendations": [
    {
      "id": 123,
      "name": "디올 소바쥬",
      "similarity": 95,
      "matchReason": "당신의 #[우디] 아우라와...",
      "details": { "topNotes": "...", "middleNotes": "...", ... }
    }
  ]
}
```

6. 안정성 및 보안 (Reliability)

- **Fallback:** 외부 AI API 장애 시 **DUMMY_RESULT** 주입 또는 MySQL 기반 계열 필터링으로 자동 전환.
- **Persistence:** 정형 데이터(MySQL)와 유연한 확장 데이터(JSON)의 하이브리드 관리 전략.